

CZECH TECHNICAL UNIVERSITY IN PRAGUE

FACULTY OF ELECTRICAL ENGINEERING
DEPARTMENT OF CYBERNETICS
MULTI-ROBOT SYSTEMS



Learning an Autonomous Drone Racing Policy via Differentiable Simulation

Master's Thesis

Adam El Ghaib

Prague, January 2026

Study programme: Cybernetics and Robotics (Space Master)

Supervisor: Ing. Robert Pěnička, Ph.D.

Abstract

Todo

Keywords Unmanned Aerial Vehicles, Autonomous Drone Racing, Simulation to Reality Transfer, Reinforcement Learning, Domain Randomization

Abbreviations

UAV Unmanned Aerial Vehicle

RL Reinforcement Learning

DR Domain Randomization

POMDP Partially Observable Markov Decision Process

DoF Degrees of Freedom

Sim2Real Simulation-to-Reality

CTBR Collective Thrust and Body Rates

ADR Autonomous Drone Racing

OCP Optimal Control Problem

OCPs Optimal Control Problems

PMP Pontryagin's Maximum Principle

SRT Single Rotor Thrust

CTBR Collective Thrust and Body Rates

LVYR Linear Velocities and Yaw Rate

TWR Thrust to Weight Ratio

Contents

1	Introduction	1
1.1	Background	1
1.2	Related Work	1
1.3	Structure and Objectives	1
2	Preliminaries	2
2.1	Optimal Control Problem	2
2.1.1	General Discrete-Time OCP	2
2.1.2	Minimum-Time OCP	2
2.1.3	Simplified Minimum-Time OCP	3
2.2	Pontryagin’s Maximum Principle	3
2.3	Classical Control approaches	4
2.3.1	Differential Flatness Based Control	4
2.3.2	Model Predictive Control	4
2.4	Learning-Based approaches	4
2.4.1	Model-Free Reinforcement Learning	4
2.4.2	Model-Based Reinforcement Learning	4
3	Methodology	5
3.1	Simulation Framework	5
3.1.1	Quadrotor Model	6
3.1.2	Control Abstraction	7
3.1.3	Domain Randomization	11
3.2	Training Tasks	13
3.2.1	Trajectory Tracking	13
3.2.2	Path Progress	13
3.3	Differentiable Optimization	13
4	Results	14
4.1	Evaluation Criteria	14
4.2	Baseline Performance	14
4.3	Ablation Study	14
4.4	Transferability to MRS UAV System	14
4.5	Real World Validation	14
5	Conclusion	15
6	Future Directions	16
7	References	17

A Appendix A**18**

■ 1 Introduction

■ 1.1 Background

Give context about the problem

What is ADR?

- Navigate through a sequence of gates as fast as possible within the platform capabilities...

What are the challenges of ADR?

- **Where am I?** → Perception
challenges: limited sensory data, noisy and uncertain partial observability
- **Where do I go?** → Planning
safe efficient path planning
- **How do I get there?** → Control
Handle the non-linear underactuated dynamics to execute the planned motion under actuator limits and model uncertainties

■ 1.2 Related Work

Present the state of the art, how do the different research labs approach this problem

Open Questions

- How to improve sample efficiency
- Generalization to multiple tasks (how to race while avoiding obstacles, how to race against other drones)
- Transferability of the controllers under unmodeled effects (sim2real transfer)

■ 1.3 Structure and Objectives

■ 2 Preliminaries

This chapter covers important theoretical aspects on which this thesis is constructed. Section 2.1 formulates the Optimal Control Problem (OCP) of drone racing and presents a theoretical solution based on the application of Pontryagin's Maximum Principle (PMP). The optimal solution derived from PMP serves primarily as an upper theoretical bound, as many of its underlying assumptions do not hold in real-world scenarios. Practical implementations must account for model uncertainties, disturbances, and system constraints that deviate from the idealized formulation.

In this context, Sections 2.3 and 2.4 review the state-of-the-art control approaches used to achieve high-performance drone racing, covering both classical control techniques and modern learning-based methods.

■ 2.1 Optimal Control Problem

cite hurak, or well...some OC textbook

■ General Discrete-Time OCP

In general, a nonlinear discrete-time OCP can be formulated as the optimization of a running cost (or loss) function $L_k()$, plus a terminal cost function $\phi()$, subject to the system's dynamics $\mathbf{f}_k()$, and the set of permitted states $\mathcal{X}_k$, and controls $\mathcal{U}_k$.

$$\begin{aligned} & \underset{\mathbf{X}, \mathbf{U}}{\text{minimize}} && \left(\phi(\mathbf{x}_N, N) + \sum_{k=0}^{N-1} L_k(\mathbf{x}_k, \mathbf{u}_k) \right) \\ & \text{subject to} && \mathbf{x}_{k+1} = \mathbf{f}_k(\mathbf{x}_k, \mathbf{u}_k), \quad k = 0, \dots, N-1, \\ & && \mathbf{u}_k \in \mathcal{U}_k, \quad k = 0, \dots, N-1, \\ & && \mathbf{x}_k \in \mathcal{X}_k, \quad k = 0, \dots, N, \end{aligned} \tag{2.1}$$

Where

N is the final discrete time,
 $\mathbf{x}_k \in \mathbb{R}^n$ is the state at the discrete time $k \in \mathbb{Z}$,
 $\mathbf{u}_k \in \mathbb{R}^m$ is the control at the discrete time $k \in \mathbb{Z}$,
 $\mathbf{X} = [\mathbf{x}_0, \dots, \mathbf{x}_N]$ is the trajectory of states,
 $\mathbf{U} = [\mathbf{u}_0, \dots, \mathbf{u}_{N-1}]$ is the trajectory of control actions.

■ Minimum-Time OCP

This last formulation of an OCP can be further specified for the case of drone racing in several manners. Since the main objective is to complete a lap through the racing track as fast as possible, this problem can be written as minimum-time problem.

$$\begin{aligned}
& \underset{\mathbf{U}, N}{\text{minimize}} && \sum_{k=0}^{N-1} 1 \\
& \text{subject to} && \mathbf{x}_{k+1} = \mathbf{f}_k(\mathbf{x}_k, \mathbf{u}_k), \quad k = 0, \dots, N-1, \\
& && \mathbf{u}_k \in \mathcal{U}_k, \quad k = 0, \dots, N-1, \\
& && \mathbf{x}_k \in \mathcal{X}_k, \quad k = 0, \dots, N,
\end{aligned} \tag{2.2}$$

The racing track restrictions, i.e. passing through a sequence of gates while avoiding obstacles, can be introduced as hard constraints under the set of permitted states $\mathcal{X}_k$ as

$$\mathcal{X}_k = \{ \mathbf{x}_k \in \mathbb{R}^n \mid \mathbf{p}_k \in \mathcal{G}_{g(k)}, \quad \mathbf{p}_k \notin \mathcal{O}_j, \quad \forall j \in \{1, \dots, M\} \}, \tag{2.3}$$

where

$\mathbf{p}_k \in \mathbb{R}^3$ is the position of the drone at time step k ,
 $\mathcal{G}_{g(k)}$ is the feasible region defined by gate g at time step k , with $g(k) \in \{1, \dots, G\}$ denoting the index of the gate to be passed at k ,
 G is the total number of gates in the racing track,
 $\mathcal{O}_j$ is the region occupied by the j -th obstacle,
 M is the total number of obstacles.

■ Simplified Minimum-Time OCP

The anterior problem formulation (subsection 2.1.2), is rather complex. It has to solve simultaneously for an optimal trajectory that passes through all the gates and avoids obstacles. And, find the optimal control sequence achivable by the drone given its dynamics and actuator constrains. To simplify this problem, throughout this thesis, either an optimal trajectory or an optimal path will be assumed to be given by an external source. Thereafter, the Optimal Control Problems (OCPs) that we adress are

Trajectory Tracking OCP

$$\begin{aligned}
& \underset{\mathbf{U}}{\text{minimize}} && \sum_{k=0}^{N-1} \left\| \mathbf{x}_k - \mathbf{x}_k^{\text{ref}} \right\|_{\mathbf{Q}}^2 \\
& \text{subject to} && \mathbf{x}_{k+1} = \mathbf{f}_k(\mathbf{x}_k, \mathbf{u}_k), \quad k = 0, \dots, N-1, \\
& && \mathbf{u}_k \in \mathcal{U}_k, \quad k = 0, \dots, N-1, \\
& && \mathbf{x}_k \in \mathcal{X}_k, \quad k = 0, \dots, N,
\end{aligned} \tag{2.4}$$

Path Progress OCP

■ 2.2 Pontryagin's Maximum Principle

maybe solve it using a pmm and Pontryagin's Maximum Principle

comment on: Since it is impossible to perfectly model the state transition equation, and the state estimation is not perfect, the optimal solution breaks under real world conditions.

■ **2.3 Classical Control approaches**

■ Differential Flatness Based Control

■ Model Predictive Control

■ **2.4 Learning-Based approaches**

■ Model-Free Reinforcement Learning

■ Model-Based Reinforcement Learning

3 Methodology

TODO Review

This chapter presents the methodology used in this thesis to train the neural network policies. In section 3.1, we address the core elements of the simulation architecture, define the quadrotor model, and explain how the Simulation-to-Reality (Sim2Real) gap is minimized by using Domain Randomization (DR). Furthermore, we introduce the implemented control abstraction levels. Section 3.2, outlines the training objectives used for Autonomous Drone Racing (ADR). Lastly, section 3.3 details how differentiability is leveraged to optimize the policies in order to minimize the training task's cost function.

3.1 Simulation Framework

Make cool diagram

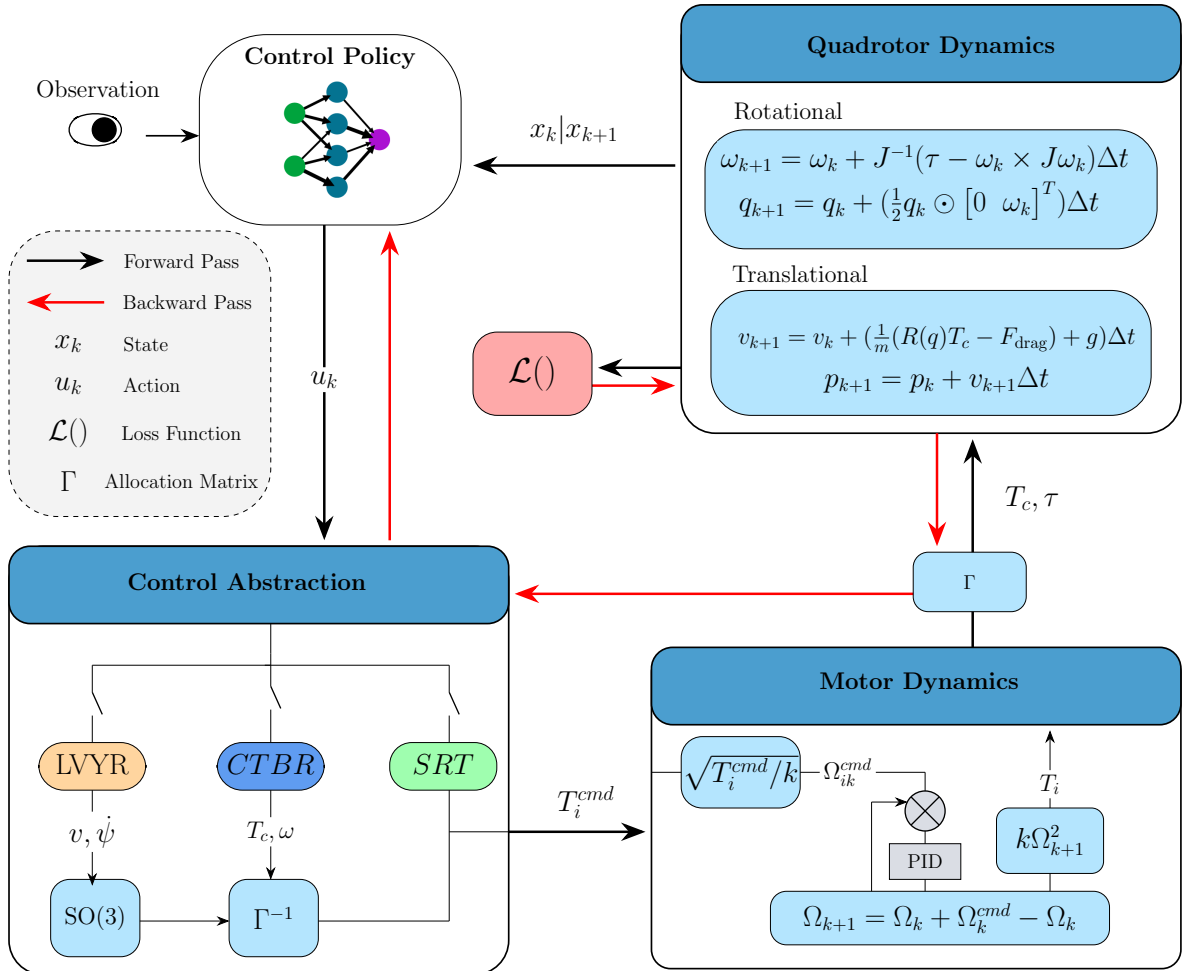


Figure 3.1: Your Caption

■ Quadrotor Model

In the simulated environments, the quadcopter is modeled as a 6 Degrees of Freedom (DoF) rigid body. The inertial world frame $\mathcal{W}$ is defined by the fixed orthonormal basis $\{\hat{\mathbf{e}}_1, \hat{\mathbf{e}}_2, \hat{\mathbf{e}}_3\}$ while the non-inertial body-fixed frame $\mathcal{B}$ by the orthonormal basis $\{\hat{\mathbf{b}}_1, \hat{\mathbf{b}}_2, \hat{\mathbf{b}}_3\}$, which is attached to the vehicle's center of mass. The state of the Unmanned Aerial Vehicle (UAV) is given by $\mathbf{x} = [\mathbf{p}, \mathbf{v}, \mathbf{q}, \boldsymbol{\omega}, \boldsymbol{\Omega}]^T$ where the position $\mathbf{p} \in \mathbb{R}^3$, velocity $\mathbf{v} \in \mathbb{R}^3$, and acceleration are represented in the world frame $\mathcal{W} \in \mathbb{R}^3$. Conversely, the body rates $\boldsymbol{\omega} \in \mathbb{R}^3$ are represented in the body frame $\mathcal{B} \in \mathbb{R}^3$ and $\boldsymbol{\Omega} \in \mathbb{R}^4$ is a vector of scalar values indicating the rotor speeds. The relationship between the two frames is given by the rotation mapping $\mathbf{b}_i = \mathbf{R}\mathbf{e}_i$, where $\mathbf{R} \in SO(3)$ is the rotation matrix corresponding to the orientation $\mathbf{q}$.

Thus, the kinematics and dynamics of the quadcopter are:

$$\dot{\mathbf{p}}^{\mathcal{W}} = \mathbf{v}^{\mathcal{W}} \quad (3.1)$$

$$\dot{\mathbf{q}} = \frac{1}{2} \mathbf{q} \odot \begin{bmatrix} 0 \\ \boldsymbol{\omega}^{\mathcal{B}} \end{bmatrix} \quad (3.2)$$

$$\dot{\mathbf{v}}^{\mathcal{W}} = \frac{\mathbf{R}(\mathbf{q})}{m} (\mathbf{F}_T^{\mathcal{B}} + \mathbf{F}_D^{\mathcal{B}}) + \mathbf{g}^{\mathcal{W}} \quad (3.3)$$

$$\dot{\boldsymbol{\omega}}^{\mathcal{B}} = \mathbf{J}^{-1} (\boldsymbol{\tau}^{\mathcal{B}} - \boldsymbol{\omega}^{\mathcal{B}} \times \mathbf{J}\boldsymbol{\omega}^{\mathcal{B}}) \quad (3.4)$$

$$\dot{\boldsymbol{\Omega}} = \frac{1}{k_m} (\boldsymbol{\Omega}_{\text{cmd}} - \boldsymbol{\Omega}) \quad (3.5)$$

In these expressions, $\odot$ denotes the quaternion multiplication operator, $\mathbf{R}(\mathbf{q})$ represents the rotation matrix mapping the body frame $\mathcal{B}$ to the world frame $\mathcal{W}$. The vectors $\mathbf{F}_T^{\mathcal{B}}$ and $\mathbf{F}_D^{\mathcal{B}}$ correspond to the collective thrust and drag forces, respectively. Finally, $\mathbf{J}$ is the diagonal inertia matrix, k_m is the motor time constant, and $\mathbf{g}$ is the gravity vector.

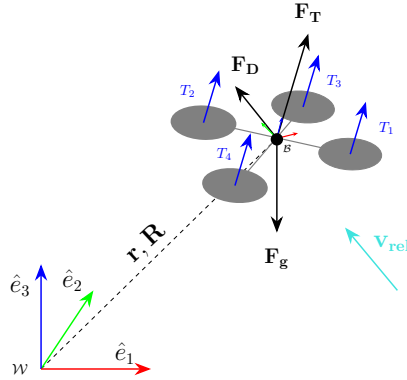


Figure 3.2: Diagram of the forces acting on the quadrotor

In the body frame $\mathcal{B}$, the individual motor thrusts T_i , $i = 1, \dots, 4$, are assumed to be perpendicular to the $\{\hat{\mathbf{b}}_1, \hat{\mathbf{b}}_2\}$ plane. The magnitudes of collective thrust F_T , and body torques $\boldsymbol{\tau}^{\mathcal{B}}$, can be obtained from the individual motor thrusts with the allocation matrix Γ .

$$\begin{bmatrix} F_T \\ \tau_{b1} \\ \tau_{b2} \\ \tau_{b3} \end{bmatrix} = \overbrace{\begin{bmatrix} 1 & 1 & 1 & 1 \\ -l \sin \beta & l \sin \beta & l \sin \beta & -l \sin \beta \\ -l \cos \beta & l \cos \beta & -l \cos \beta & l \cos \beta \\ -c_{tf} & -c_{tf} & c_{tf} & c_{tf} \end{bmatrix}}^{\Gamma} \begin{bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \end{bmatrix} \quad (3.6)$$

Where l is the arm length and c_{tf} the linear torque constant. The individual motor thrusts T_i , can be obtained from the rotor speeds Ω_i with the thrust map coefficients, $\mathbf{k}_T = [a_0, a_1, a_2]$ which can be obtained experimentally by fitting a quadratic curve to the rotor speed versus thrust data.

$$T_i = \mathbf{k}_T \begin{bmatrix} \Omega_i^2 \\ \Omega_i \\ 1 \end{bmatrix} = [a_0 \ a_1 \ a_2] \begin{bmatrix} \Omega_i^2 \\ \Omega_i \\ 1 \end{bmatrix} \quad (3.7)$$

For convenience, we simplicate Equation 3.7 to $T_i = a_0 \Omega_i^2$.

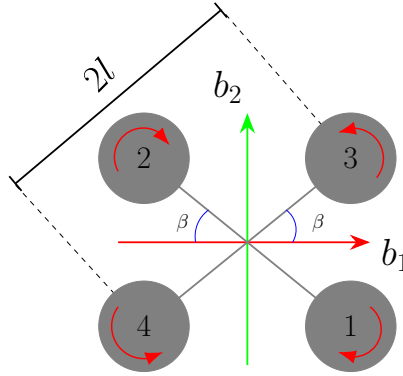


Figure 3.3: Diagram of the forces acting on the quadrotor

■ Control Abstraction

The control abstraction refers to the modality of the policy outputs used to control the quadrotor. The choice of control abstraction affects both the performance of the controller for a given task and the sample efficiency during training. For this reason, it is worthwhile to investigate and implement multiple control abstractions.

In this thesis, three control modalities are considered. The policy output layer employs a sigmoid activation function for all control modalities, ensuring that the action space is bounded, which facilitates stable learning. The lowest level of abstraction consists of directly outputting Single Rotor Thrust (SRT) for each motor. An intermediate level of abstraction outputs the collective thrust and the desired body rates, which are subsequently mapped to individual rotor thrusts through a control allocation matrix. The highest level of abstraction outputs desired linear velocities and yaw rate, which are converted into a wrench using an $SO(3)$ geometric controller **TODO : Cite Lee**, and subsequently mapped to rotor thrusts in a manner analogous to the collective thrust and body rate formulation. The mathematical formulation of these transformations is presented next.

Single Rotor Thrust

When the SRT control mode is chosen, the only transformation to be applied is scaling the policy output $\mathbf{u} \in [0, 1] \in \mathbb{R}^4$ to the maximum thrust that can be produced by the motors, $T_{\max}$. Therefore, the transformation is as simple as:

$$\mathbf{T}_{\text{cmd}} = \mathbf{u}T_{\text{max}} \quad (3.8)$$

TODO : Add min thrust mapping xd

Once the scaling is applied, to better emulate the behaviour of a real motor, a first order model with rotor speed saturation is implemented. First, the commanded motor thrust $\mathbf{T}_{\text{cmd}}$ are converted to commanded rotor speed. This mapping can be obtained experimentally by fitting a quadratic curve to the experimental data. In most cases the following mapping approximation is good enough.

$$\mathbf{T}_{\text{cmd}} \approx k\boldsymbol{\Omega}_{\text{cmd}}^2 \quad (3.9)$$

Then the commanded rotor speeds $\boldsymbol{\Omega}_{\text{cmd}}$, are passed through the following first order motor model Equation 3.10, and saturated according to the maximum rotor speed rate, $\dot{\Omega}_{\text{cmd}}$ given by the manufacturer.

$$H(s) = \frac{1}{k_m + s} \quad (3.10)$$

Finally, the rotor speeds obtained after the described process are mapped again to single rotor thrust values using Equation 3.9 and then converted to collective force and torques with Equation 3.11, to calculate the next state through the quadrotor dynamics model.

$$\begin{bmatrix} T_c \\ \tau_1 \\ \tau_2 \\ \tau_3 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \end{bmatrix} \quad (3.11)$$

Modeling the motor dynamics presents more realistic data which the policy will encounter at deployment. However, for training purposes it has been decided to skip the motor dynamics during the backpropagation pass as it significantly destabilizes the gradients computation. Particularly, the mapping of Equation 3.9 produces high variations on the gradient, degrading the training performance and sample efficiency.

Collective Thrust and Body Rates

When Collective Thrust and Body Rates (CTBR) control mode is selected, the normalized outputs of the policy $\mathbf{u} \in [0, 1] \in \mathbb{R}^4$, are mapped to collective thrust and body rates. The scaling is as follows,

$$T_{c,\text{cmd}} = u_1(T_{\text{max}} - T_{\text{min}}) + T_{\text{min}} \quad (3.12)$$

$$\omega_{i,\text{cmd}} = (2u_{i+1} - 1)\omega_{i,\text{max}}, \quad i \in \{1, 2, 3\} \quad (3.13)$$

where $\omega_{i,\text{max}}$ indicates the maximum body rate of the respective axis, (roll, pitch, yaw). Once the rates are calculated, they can be converted to torque values,

$$\boldsymbol{\tau}_{\text{cmd}} = k_p \frac{(\boldsymbol{\omega}_{\text{cmd}} - \boldsymbol{\omega})}{dt} J \quad (3.14)$$

additionally, a proportional gain k_p can be introduced to adjust the stiffness of the conversion. Given the commanded collective thrust and torques, the motor thrusts can be obtained from Equation 3.11 and then the same procedure of subsection 3.1.2 is applied during the forward and backward passes.

Linear Velocities and Yaw Rate

The highest level of abstraction that is implement consists on commanding Linear Velocities and Yaw Rate (LVYR). With this control mode, the policy outputs are scaled as,

$$v_{i,\text{cmd}} = (2u_i - 1)v_{i,\text{max}}, \quad i \in \{1, 2, 3\} \quad (3.15)$$

$$\omega_{3,\text{cmd}} = \dot{\psi}_{\text{cmd}} = (2u_4 - 1)\dot{\psi}_{\text{max}} \quad (3.16)$$

where ψ denotes the yaw angle and $v_{i,\text{cmd}}$ represents the commanded linear velocity along each body axis. Mapping these control commands to thrust commands is less straightforward than in the previous two methods. Instead, the collective thrust and torques are obtained by applying a transformation in the Special Orthogonal group $\text{SO}(3)$. The transformations proceed in the following manner.

First, the acceleration vector is computed as

$$\mathbf{a}_{\text{cmd}} = k_v(\mathbf{v}_{\text{cmd}} - \mathbf{v}) + g\hat{\mathbf{b}}_3 \quad (3.17)$$

where k_v is a proportional gain on the velocity. Note that the acceleration along the third body axis is compensated for gravity; $\hat{\mathbf{b}}_3$ denotes the unit vector along this axis. Then, force vector is the UAV's mass times the commanded acceleration vector, and the commanded collective thrust vector is the projection of the force vector onto the $\hat{\mathbf{b}}_3$ axis.

$$\mathbf{F} = m\mathbf{a}_{\text{cmd}} \quad (3.18)$$

$$T_{c,\text{cmd}} = \mathbf{F} \cdot \hat{\mathbf{b}}_3 \quad (3.19)$$

To obtain the commanded bodytorques, the heading vector is obtained by projecting the yaw angle into the world plane defined by $\text{span}\{\hat{\mathbf{e}}_1, \hat{\mathbf{e}}_2\}$. The heading vector, serves an auxiliary vector to compute $\hat{\mathbf{b}}_2$ and $\hat{\mathbf{b}}_1$. **TODO Double check whether projecting the heading angle is actually correct**

$$\mathbf{h} = [\cos \psi \quad \sin \psi \quad 0]^T \quad (3.20)$$

$$\hat{\mathbf{b}}_2 = \frac{\hat{\mathbf{b}}_3 \times \mathbf{h}}{\|\hat{\mathbf{b}}_3 \times \mathbf{h}\|} \quad (3.21)$$

$$\hat{\mathbf{b}}_1 = \hat{\mathbf{b}}_2 \times \hat{\mathbf{b}}_3 \quad (3.22)$$

In this way, the commanded orientation is defined by $\mathbf{R}_{\text{cmd}} = [\hat{\mathbf{b}}_1 \quad \hat{\mathbf{b}}_2 \quad \hat{\mathbf{b}}_3]$ and the error between the commanded orientation and the current orientation is computed as in [Lee].

$$\mathbf{e}_R = \frac{1}{2} (\mathbf{R}_{\text{cmd}}^T \mathbf{R} - \mathbf{R}^T \mathbf{R}_{\text{cmd}})^\vee \quad (3.23)$$

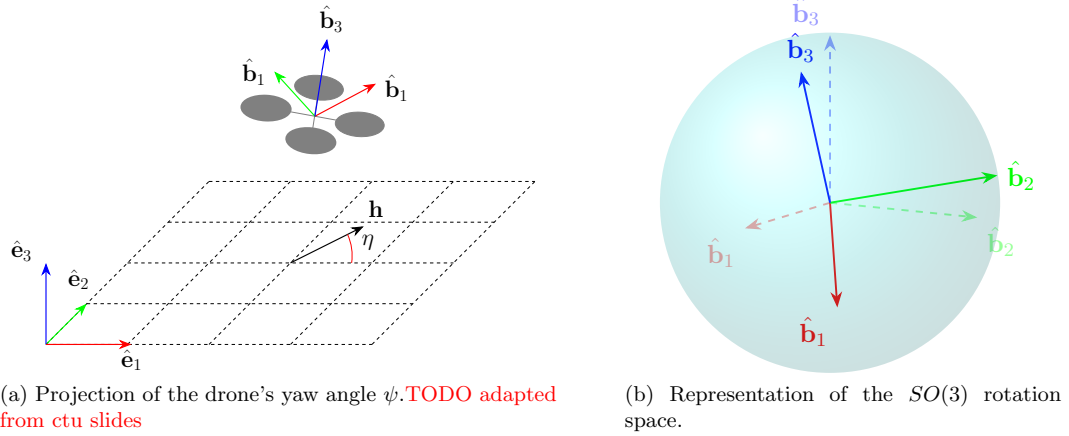


Figure 3.4: Geometric interpretation of Equations 3.12–3.15

The complete derivation of Equation 3.23 can be found in the original paper, [Lee]. To provide intuition, note that if $\mathbf{R}_{\text{cmd}} = \mathbf{R}$, then $\mathbf{R}_{\text{cmd}}^T \mathbf{R} = \mathbf{I}$ and the error is zero. Otherwise, the matrix $\mathbf{R}_{\text{cmd}}^T \mathbf{R}$ represents the relative rotation between the current and desired orientations. The skew-symmetric part of this matrix, given by $\frac{1}{2}(\mathbf{R}_{\text{cmd}}^T \mathbf{R} - \mathbf{R}^T \mathbf{R}_{\text{cmd}})$, encodes the axis and magnitude of the rotation error. The vee map $(\cdot)^\vee$ then converts this skew-symmetric matrix into a vector in $\mathbb{R}^3$, corresponding to the rotation error expressed in the Lie algebra $\mathfrak{so}(3)$, that is to say, $^\vee: \mathfrak{so}(3) \rightarrow \mathbb{R}^3$. Then, the factor $\frac{1}{2}$ ensures consistency with the standard definition of the vee operator and avoids double counting of the off-diagonal terms.

Additionally, an error on the angular velocity of the rotation is introduced as,

$$\mathbf{e}_\omega = \omega - \mathbf{R}^T \mathbf{R}_{\text{cmd}} \omega_{\text{cmd}} \quad (3.24)$$

where $\omega_{\text{cmd}} = [0 \ 0 \ \dot{\psi}_{\text{cmd}}]^T$. Note that the error between the desired rotational velocity and the current rotational velocity line on different tangent planes to the unit sphere, $\dot{\mathbf{R}} \in T_{\mathbf{R}}SO(3)$ and $\dot{\mathbf{R}}_{\text{cmd}} \in T_{\mathbf{R}_{\text{cmd}}}SO(3)$. For this reason, as explained in [Lee], $\dot{\mathbf{R}}_{\text{cmd}}$ is transformed into a vector in the $T_{\mathbf{R}}SO(3)$ space to be able to compare it to $\dot{\mathbf{R}}$ and obtain the error; Equation 3.24. Finally, the torque is set by,

$$\tau_{\text{cmd}} = -k_R \mathbf{e}_R - k_\omega \mathbf{e}_\omega + \omega \times \mathbf{J} \omega - \mathbf{J}(\hat{\omega} \mathbf{R}^T \mathbf{R}_{\text{cmd}} \omega_{\text{cmd}} - \mathbf{R}^T \mathbf{R}_{\text{cmd}} \dot{\omega}_{\text{cmd}}) \quad (3.25)$$

where a gyroscopic and feed forward terms are added to the rotation and rotational speed errors. The *hat map* $\hat{\cdot}: \mathbb{R}^3 \rightarrow \mathfrak{so}(3)$ is defined by the condition that $\hat{x}y = x \times y$ for all $x, y \in \mathbb{R}^3$. In this manner, the commanded collective thrust Equation 3.19, and commanded torques Equation 3.25, can be mapped to single rotor thrust commands to apply the same procedure of the last control modes during the forward and backward passes.

Linear Velocity Yaw Rate and Gains

Finally, the last control mode implemented is an augmentation of the LVYR controller. In this formulation, the policy outputs high-level commands in the form of desired linear velocities and yaw rate, rather than directly commanding individual rotor thrusts. This higher level of abstraction simplifies the learning process, as the policy does not need to implicitly learn how differential rotor thrusts generate torques, since this mapping is handled by the geometric controller. As shown in ?? **TODO cite the section**, this abstraction enables faster learning. However, this approach does not necessarily minimize the loss function, as the policy performance can be significantly constrained by the tuning of the controller gains. For this reason, it is of interest to investigate how performance is affected when the policy is allowed to output the controller gains in addition to the velocity and yaw rate commands.

Thus, when this control mode is selected, the policy outputs are, the same as Equation 3.15, Equation 3.24, and the gains $\{k_v, k_R, k_\omega\}$ are included by simply scaling the policy outputs to the maximum allowed gains.

$$k_{v,cmd} = u_5 k_{v,max} \quad (3.26)$$

$$k_{R,cmd} = u_6 k_{R,max} \quad (3.27)$$

$$k_{\omega,cmd} = u_7 k_{\omega,max} \quad (3.28)$$

■ Domain Randomization

To improve the transferability of the policy and to prevent the it from overfitting to a specific environment, DR is implemented in our methodology as it is well-stabilshed strategy in the field, **TODO : Cite**

A domain can be defined as a specific subset of all possible Partially Observable Markov Decision Process (POMDP) environments. While a complete POMDP is formally characterized by an 8-tuple [1], we focus on a reduced 6-tuple representation that captures the essential elements for our domain definition. A standard domain is then characterized by $\mathcal{D} = (\mathcal{S}, \mathcal{A}, \mathcal{O}, P, O, R)$ where:

- $\mathcal{S}$: denotes a set of states called the state space.
- $\mathcal{A}$: is the set of actions called the action space.
- $\mathcal{O}$: is the set of observations called the observation space.
- $\mathbf{P}(\mathbf{s}'|\mathbf{s}, \mathbf{a})$: is the probability that a given action $a \in \mathcal{A}$, at state $s \in \mathcal{S}$ will lead to a new state $s' \in \mathcal{S}$.
- $\mathbf{O}(\mathbf{o}|\mathbf{s}', \mathbf{a})$: is the observation probability, representing the probability of observing $o \in \mathcal{O}$ after taking action a and transitioning to state s' .
- $\mathbf{R}(\mathbf{s}', \mathbf{s}, \mathbf{a})$: is the immediate reward, or expected immediate reward, received for transitioning from s to s' after performing action a .

From the source domain, $\mathcal{D}_{\xi_s}$, (i.e. simulation), we can control a set of N parameters that define a configuration vector, $\boldsymbol{\xi} = [\xi_i, \xi_{i+1}, \dots, \xi_N]$. These parameters will affect the transition probability $\mathbf{P}(\mathbf{s}'|\mathbf{s}, \mathbf{a})$ of the domain. Examples of ξ_i can include:

Physical properties	$\xi_{\text{mass}}, \xi_{\text{inertia}}, \xi_{\text{drag}}, \xi_{\text{gravity}}$
Perception properties	$\xi_{\text{sensor_noise}}, \xi_{\text{sensor_latency}}$
Viual properties	$\xi_{\text{lighting}}, \xi_{\text{texture}}, \xi_{\text{fov}}$
Scene properties	$\xi_{\text{gate_pos}}, \xi_{\text{gate_size}}, \xi_{\text{gate_num}}$

When DR is applied, each configuration ξ is sampled from a bounded randomization space Ξ with a probability distribution. That is to say, $\xi \in \Xi \subset \mathbb{R}^N$, $\xi \sim p(\xi)$ where $\xi_i \in [\xi_i^{\text{low}}, \xi_i^{\text{high}}]$. At the start of every training episode, a new environment instance is created by sampling a new configuration vector ξ from the distribution $p(\xi)$. Therefore, the policy will interact with a new POMDP environment, a new domain $\mathcal{D}_\xi$. This way, data from a variety of environments that otherwise would remain unexplored can be collected.

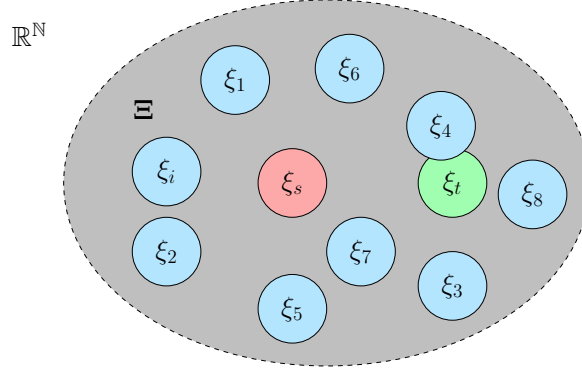


Figure 3.5: In red, the source domain. In light blue, randomized instances of the source configuration. In green, the target domain configuration.

Hereby, during training, the neural network weights and biases θ are optimized to maximize the expected reward R /**TODO :minimize the loss** across a distribution of randomized domains [2]. With the aim of finding the optimal, θ^* that will lead to the optimal policy π_{θ}^* .

As illustrated in Figure 3.5 and described in ??, the policy is exposed to a variety of domains to promote generalization and improve robustness, increasing the likelihood of successful transfer to the real environment. However, this strategy further reduces the sample efficiency of Reinforcement Learning (RL) and increases training time. Therefore, it is essential to properly bound the randomization space Ξ , so that it encompasses only the minimum necessary range to include the target domain.

In order to efficiently bound this randomization space while still allowing for enough randomness a two stage randomization is implemented. Inspired by **TODO :Cite, Zhang**, first a design informed randomization **TODO :Add acronym?** is applied. This ensures that sensible variations of the nominal platform are predominant in the randomization space. Thus avoiding that unfeasible designs such as a quadrotor with unsensible Thrust to Weight Ratio (TWR) (i.e. $\text{TWR} < 1$). The authors of [**empty citation**] notice that quadrotor designs, in general, follow a similar pattern where the quadrotors armlength l is strongly correlated to its mass m , inertia $\mathbf{J}$, drag coefficients $\mathbf{c}_d$ and other physical parameters.

Hence, they propose to sample a size factor $c \sim \mathcal{U}(0, 1)$ and scale the quadrotor's physical parameters as listed below;

$$l = c(l_{\max} - l_{\min}) + l_{\min} \quad (3.29)$$

$$m = \frac{l^3 - l_{\min}^3}{l_{\max}^3 - l_{\min}^3} (m_{\max} - m_{\min}) + m_{\min} \quad (3.30)$$

$$\mathbf{J} = \frac{l^5 - l_{\min}^5}{l_{\max}^5 - l_{\min}^5} (\mathbf{J}_{\max} - \mathbf{J}_{\min}) + \mathbf{J}_{\min} \quad (3.31)$$

$$\mathbf{c}_d = \frac{l^2 - l_{\min}^2}{l_{\max}^2 - l_{\min}^2}(\mathbf{c}_{d_{\max}} - \mathbf{c}_{d_{\min}}) + \mathbf{c}_{d_{\min}} \quad (3.32)$$

TODO :Add the rest of the params

Following the scaling process, these parameters which comprise the following configuration vector

$$\boldsymbol{\xi} = [l \quad m \quad \mathbf{J} \quad \mathbf{c}_d] \quad (3.33)$$

are subjected to additional noise. Each parameter is multiplied by a scaling factor $\eta \sim \mathcal{U}(1 - p, 1 + p)$, where $p = 0.2$. This specific uncertainty range was selected based on experimental results from a previous project, as it demonstrated good transferability, nonetheless it can be considered a hyperparameter of the training process and be tuned according to the situation.

■ 3.2 Training Tasks

■ Trajectory Tracking

■ Path Progress

■ 3.3 Differentiable Optimization

■ 4 Results

- 4.1 Evaluation Criteria
- 4.2 Baseline Performance
- 4.3 Ablation Study
- 4.4 Transferability to MRS UAV System
- 4.5 Real World Validation

5 Conclusion

■ 6 Future Directions

■ 7 References

- [1] K.J Åström. “Optimal control of Markov processes with incomplete state information”. In: *Journal of Mathematical Analysis and Applications* 10.1 (1965), pp. 174–205. ISSN: 0022-247X. DOI: [https://doi.org/10.1016/0022-247X\(65\)90154-X](https://doi.org/10.1016/0022-247X(65)90154-X). URL: <https://www.sciencedirect.com/science/article/pii/0022247X6590154X>.
- [2] Lilian Weng. “Domain Randomization for Sim2Real Transfer”. In: *lilianweng/github/io* (2019). URL: <https://lilianweng.github.io/posts/2019-05-05-domain-randomization/>.

■ A Appendix A